

1

The screenshot shows the LeetCode interface for problem 9, "Palindrome Number". The problem description states: "Given an integer x , return `true` if x is a *palindrome*, and `false` otherwise." It includes three examples: Example 1 (x=121, true), Example 2 (x=-121, false), and Example 3 (x=10, false). Constraints specify $-2^{31} \leq x \leq 2^{31} - 1$. A follow-up question asks if the problem can be solved without converting the integer to a string. The code editor shows a C++ solution using a while loop to reverse the digits of the number. The test results show the solution is "Accepted" with a runtime of 0 ms.

9. Palindrome Number

Easy Topics Companies Hint

Given an integer x , return `true` if x is a *palindrome*, and `false` otherwise.

Example 1:
 Input: $x = 121$
 Output: `true`
 Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:
 Input: $x = -121$
 Output: `false`
 Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:
 Input: $x = 10$
 Output: `false`
 Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Constraints:

- $-2^{31} \leq x \leq 2^{31} - 1$

Follow up: Could you solve it without converting the integer to a string?

Seen this question in a real interview before? 1/5

Accepted 7,795,221 / 22,984 Acceptance Rate 60.3%

Topics Companies

15.7K 695 320 Online

```

1 class Solution {
2 public:
3     bool isPalindrome(int x) {
4         if (x < 0 || (x % 10 == 0 && x != 0)) {
5             return false;
6         }
7
8         int reversed = 0;
9         while (x > reversed) {
10             reversed = reversed * 10 + x % 10;
11             x /= 10;
12         }
13         return x == reversed || x == reversed / 10;
14     }
15 }

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

121

Output

true

Expected

true

Contribute a testcase

2

The screenshot shows the LeetCode interface for problem 10, "Regular Expression Matching". The problem description states: "Given an input string s and a pattern p , implement regular expression matching with support for `.` and `*` where:

- `.` Matches any single character.
- `*` Matches zero or more of the preceding element.

 Return a boolean indicating whether the matching covers the entire input string (not partial)." It includes three examples: Example 1 (s="aa", p="a", false), Example 2 (s="aa", p="a*", true), and Example 3 (s="ab", p=".*k", true). Constraints specify $1 \leq s.length \leq 20$, $1 \leq p.length \leq 20$, s contains only lowercase English letters, p contains only lowercase English letters, `.`, and `*`, and it is guaranteed for each appearance of the character `*`, there will be a previous valid character to match. The code editor shows a C++ solution using dynamic programming. The test results show the solution is "Accepted" with a runtime of 0 ms.

10. Regular Expression Matching

Hard Topics Companies

Given an input string s and a pattern p , implement regular expression matching with support for `.` and `*` where:

- `.` Matches any single character.
- `*` Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:
 Input: $s = "aa"$, $p = "a"$
 Output: `false`
 Explanation: "a" does not match the entire string "aa".

Example 2:
 Input: $s = "aa"$, $p = "a^*$
 Output: `true`
 Explanation: `a^*` means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:
 Input: $s = "ab"$, $p = ".*k"$
 Output: `true`
 Explanation: `.*k` means "zero or more (*) of any character (.)".

Constraints:

- $1 \leq s.length \leq 20$
- $1 \leq p.length \leq 20$
- s contains only lowercase English letters.
- p contains only lowercase English letters, `.`, and `*`.
- It is guaranteed for each appearance of the character `*`, there will be a previous valid character to match.

Seen this question in a real interview before? 1/5

Accepted 13,2K 528 178 Online

```

1 for (int j = 1; j <= m; j++) {
2     if (p[j-1] == ".") {
3         dp[j][j] = dp[j][j-1];
4     }
5 }
6
7 for (int i = 1; i <= m; i++) {
8     for (int j = 1; j <= m; j++) {
9         if (p[j-1] == ".") {
10             dp[i][j] = dp[i][j-1];
11         }
12         else if (p[j-1] == "*" || p[j-1] == s[i-1]) {
13             dp[i][j] = dp[i][j-1] || dp[i-1][j];
14         }
15         else if (p[j-1] == "*" || p[j-1] == s[i-1]) {
16             dp[i][j] = dp[i-1][j-1];
17         }
18     }
19 }
20 return dp[m][m];

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s = "aa"

p = "a"

Output

false

Expected

false

3

Yo what Speaking Club Web CN Information Educational MDAI IST Physics OOP

leetcode.com/problems/container-with-most-water/

11. Container With Most Water

Medium Tests Comments Help

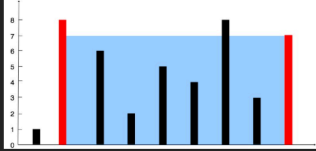
You are given an integer array `height` of length n . There are n vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Inputs: `height = [1,8,6,2,5,4,6,3,7]`
Output: 49
Explanation: The above vertical lines are represented by array `[1,8,6,2,5,4,6,3,7]`. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: `height = [1,1]`
Output: 1

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 10^5$
- $0 \leq \text{height}[i] \leq 10^5$

Seen this question in a real interview before? 1/5

33.9K 660 445 Online

```

1 class Solution {
2 public:
3     int maxArea(vector<int>& height) {
4         int left = 0;
5         int right = height.size() - 1;
6         int maxwater = 0;
7
8         while (left < right) {
9             int width = right - left;
10            int h = min(height[left], height[right]);
11            maxwater = max(maxwater, width * h);
12
13            if (height[left] < height[right]) {
14                left++;
15            } else {
16                right--;
17            }
18        }
19
20        return maxwater;
21    }
22 };

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`height =`

`[1,8,6,2,5,4,6,3,7]`

Output

49

Expected

49

Contribute a testcase

4

Yo what Speaking Club Web CN Information Educational MDAI IST Physics OOP

leetcode.com/problems/integer-to-roman/

Integer to Roman

Medium Tests Comments Help

Given an integer, convert it to a Roman numeral.

Example 1:

Input: `num = 3749`
Output: "MMMDCCXLIX"
Explanation:

3000 = MMM as 1000 (M) + 1000 (M) + 1000 (M)
700 = DCC as 500 (D) + 100 (C) + 100 (C)
40 = XL as 10 (X) less of 50 (L)
9 = IX as 1 (I) less of 10 (X)
Note: 49 is not 1 (I) less of 50 (L) because the conversion is based on decimal places

Example 2:

Input: `num = 58`
Output: "LVIII"
Explanation:

50 = L
8 = VIII

Example 3:

Input: `num = 1994`
Output: "MCMXCIV"
Explanation:

1000 = M
900 = CM
90 = XC
4 = IV

Constraints:

- $1 \leq \text{num} \leq 3999$

Seen this question in a real interview before? 1/5

8.5K 221 115 Online

```

1 class Solution {
2 public:
3     string intToRoman(int num) {
4         vector<pair<int, string>> values = {
5             {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
6             {100, "C"}, {90, "XC"}, {50, "L"}, {40, "XL"},
7             {10, "X"}, {9, "IX"}, {5, "V"}, {4, "IV"}, {1, "I"}
8         };
9
10        string result = "";
11
12        for (const auto& [value, symbol] : values) {
13            while (num >= value) {
14                result += symbol;
15                num -= value;
16            }
17        }
18
19        return result;
20    }
21 };

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`num =`

`3749`

Output

"MMMDCCXLIX"

Expected

"MMMDCCXLIX"

Contribute a testcase

5

6

7

15. 3Sum

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `j != k`, and `i != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:
Input: `nums = [-1,0,1,2,-1,-4]`
Output: `[[-1,-1,2], [-1,0,1]]`
Explanation:
`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0`,
`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0`,
`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0`.
The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.
Notice that the order of the output and the order of the triplets does not matter.

Example 2:
Input: `nums = [0,1,1]`
Output: `[]`
Explanation: The only possible triplet does not sum up to 0.

Example 3:
Input: `nums = [0,0,0]`
Output: `[[0,0,0]]`
Explanation: The only possible triplet sums up to 0.

Constraints:

- `3 <= nums.length <= 3000`
- `-105 <= nums[i] <= 105`

Seen this question in a real interview before? ☐ Yes ☒ No

Accepted 5,701,786/4.8M Acceptance Rate 38.6%

35.2K 779 633 Online

```
while (left < right) {
    int sum = nums[left] + nums[right];
    if (sum == target) {
        result.push_back({nums[left], nums[right]});
        while (left < right && nums[left] == nums[left + 1]) left++;
        while (left < right && nums[right] == nums[right - 1]) right--;
        left++;
        right--;
    }
    else if (sum < target) {
        left++;
    }
    else {
        right--;
    }
}
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums =`
`[-1,0,1,2,-1,-4]`

Output

`[[-1,-1,2], [-1,0,1]]`

Expected

`[[-1,-1,2], [-1,0,1]]`

Contribute a testcase

8

16. 3Sum Closest

Given an integer array `nums` of length `n` and an integer `target`, find three integers at **distinct indices** in `nums` such that the sum is closest to `target`. Return the sum of the three integers.

You may assume that each input would have exactly one solution.

Example 1:
Input: `nums = [-1,2,1,-4], target = 1`
Output: 2
Explanation: The sum that is closest to the target is 2. `(-1 + 2 + 1 = 2)`.

Example 2:
Input: `nums = [0,0,0], target = 1`
Output: 0
Explanation: The sum that is closest to the target is 0. `(0 + 0 + 0 = 0)`.

Constraints:

- `3 <= nums.length <= 500`
- `-1000 <= nums[i] <= 1000`
- `-104 <= target <= 104`

Seen this question in a real interview before? ☐ Yes ☒ No

Accepted 1,761,786/3.7M Acceptance Rate 48.1%

Topics

Companies

Similar Questions

```
while (left < right) {
    int currentSum = nums[left] + nums[right];
    if (abs(currentSum - target) < abs(closestSum - target)) {
        closestSum = currentSum;
    }
    if (currentSum < target) {
        left++;
    }
    else if (currentSum > target) {
        right--;
    }
    else {
        return currentSum;
    }
}
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`nums =`
`[-1,2,1,-4]`

`target =`
1

Output

2

Expected

2